

열화상 주차 감지 시스템 — 실시간 CV 알고리즘 설계·구현 (Python)

바닥 → 천장 방향으로 설치한 열화상 카메라로 입출차를 실시간 감지하는 시스템의 알고리즘을 개발하고 실시간·오프라인 도구를 전담 구현하고, 온도 절대값의 한계를 NCC 패턴 매칭으로, ROI 위치마다 임계값을 다시 잡아야 하는 문제를 해결했습니다.

본 문서는 포트폴리오용 PDF입니다. 회사·배포지·SDK·인프라 식별 정보와 민감 구현 세부 (레지스터 맵, DB 스키마 등)는 제거하였습니다.

1. 프로젝트 개요

항목	내용
도메인	열화상 카메라 기반 실시간 주차면 입출차 감지
담당 역할	알고리즘 설계 + 실시간/오프라인 시스템 전담 구현
기간	약 2개월+ (알고리즘 5세대 반복 개선)
산출물	실시간 감지 GUI, 오프라인 분석 GUI(ROI 드로잉), 학습 모델 포맷, 기술 발표 자료

감지 원리: 카메라를 바닥에서 천장 방향으로 설치. 차가 있으면 차체 하부가 시야를 가리고, 없으면 천장 패턴이 보입니다. 이 **패턴의 유사도 변화**로 점유 여부를 판정합니다.

- 열화상은 시간대(아침/오후)·계절에 따라 절대 온도가 통째로 오르내리고, 노이즈가 있고, 또한 판정 영역(ROI)의 위치·질감에 따라 기준선이 달라져 카메라마다·ROI마다 임계값을 다시 잡아야 합니다.

2. 기술 스택

Python · NumPy · OpenCV · WebSocket (실시간 프레임 스트리밍) · tkinter / matplotlib (GUI·시각화) · npz (학습 reference 영속화)

3. 알고리즘 설계 — 문제·원인·해결·결과

핵심 성과는 하나의 알고리즘이 아니라, 문제를 반복적으로 진단하며 접근을 근본적으로 바꾼 과정입니다. 주요 전환점을 문제 → 해결 → 결과로 정리하였습니다.

3.1 절대 온도 판정의 실패

- **문제발생:** 온도 평균 임계값으로 점유를 판정했으나, 시간대(아침/오후)·계절에 따라 온도가 통째로 이동해 오판이 잦음.
- 분포 너비(spread)로 바뀌도 실내 천장은 원래 균일해 차체와 구분되지 않고, 근본 원인은 **절대 온도·전역 통계가 환경 변화에 그대로 노출**
- **해결방안:** 각 프레임을 평균 제거 후 표준편차로 정규화하고, reference와의 **패턴 상관도(NCC)**를 판정 신호로 채택

```
def normalize(t):  
    return (t - t.mean()) / t.std()  
  
def ncc(a_norm, b_norm):  
    return float(np.mean(a_norm * b_norm)) # 패턴 유사도  
(-1 ~ 1)
```

- **결과:** 정규화가 온도의 전역 이동을 상쇄해 온도가 통째로 올라도 NCC는 불변.

3.2 노이즈로 인한 상태 진동

- **문제발생:** NCC가 순간 노이즈로 튀면 점유/빈 상태가 진동해 **허위 입출차 이벤트**가 발생. 원인은 단일 프레임의 순간값을 그대로 상태로 확정했기 때문.
- **해결방안:** **EMPTY → ENTERING → PARKED → EXITING** 상태 머신을 도입하고, **일정 시간 연속으로 안정될 때만** 상태를 확정
- **결과:** 순간적 튜는 무시되고 실제 입출차만 이벤트로 기록되어, 이벤트 신뢰도가 크게 오름

3.3 ROI마다 임계값을 다시 잡아야 하는 문제

- **문제발생(원인):** 빈 칸인데도 NCC가 1.0까지 안 오르고 ROI마다 0.6~0.9 언저리에서 멈췄다. 이 빈-상태 값(**B**)이 질감 많은 ROI에선 높고 밋밋한 데선 낮다 보니, ROI를 옮길 때마다 임계값을 새로 잡아야 했다. 파고들어 보니 **B**는 그 ROI의 신호 대 노이즈 비(SNR)로 정해지고 있었다.
- **해결방안:** 임계값을 계속 손보는 대신, **B**가 SNR로 정해진다는 점을 거꾸로 활용했다. 학습 때 픽셀별 노이즈 σ 맵을 한 번 만들어두면 ROI 마스크만 넘겨도 그 안의 신호(**sv**)·노이즈(**nv**)로 **B**를 계산할 수 있다. 실측 NCC를 이 **B**로 나눠 **occ = 1 - NCC/B**로 바꾸니 ROI가 달라도 점수 범위가 같아졌다.

```

# 학습: 빈 구간 프레임들로 reference + 픽셀별 시간 노이즈
ref = frames.mean(0) # 평균 = reference
ac = frames - frames.mean(axis=(1,2), keepdims=True)
sigma = ac.std(0) # 공통성분 제거한 노이즈

# 판정: 어떤 ROI든 baseline B를 예측
def predict_baseline(ref, sigma, mask):
    sv = ref[mask].var() # ROI 패턴 세기
    nv = (sigma[mask]**2).mean() # ROI 시간 노이즈 세기
    return 1 / sqrt(1 + nv / sv) # SNR 기반 이론적 baseline

occ = 1 - NCC / B

```

- **결과:** 질감이 센 ROI든 밋밋한 ROI든 `occ` 로 환산하면 차이가 상쇄돼, 임계값 하나로 전부 판정할 수 있게 됐다.

4. 그 밖의 핵심 기술 의사결정

4.1 학습 reference 고정 — 자동갱신 제거

초기에는 빈 상태일 때 reference와 baseline을 서서히 자동갱신했다. 그런데 한 실내 카메라가 계속 점유(PARKED)로 갇히는 현상이 발생.

- **진단:** 학습 시점 대비 시간이 흐르며 baseline 값이 부풀어 올라(예: 2.7 → 4.8), 점유 판정 기준을 영원히 넘지 못하는 상태가 됨. 자동갱신이 **기준선 자체를 오염**시킨 것.
- **판단:** NCC가 이미 온도 변화를 흡수하므로 자동갱신은 불필요하다고 보고 **전면 제거**. 학습값을 고정하니 안정성이 오르고 코드도 단순해졌다.

이 사례는 “가정을 의심하고, 데이터(단 몇 개의 값)로 근본 원인을 진단”하는 접근의 대표 사례였다.

4.2 시간(ms) 기반 상태 머신 — FPS 독립성

안정성 검증을 **프레임 카운트**로 하면 데이터마다 FPS가 다를 때(오프라인 1fps vs 실시간 5fps) 실제 대기 시간이 달라진다. 이동평균·안정 확정·프레임 갭 판정을 모두 **물리적 시간(ms)** 기준으로 바꿔, 어떤 FPS에서도 동일하게 “2초 안정 후 확정”이 성립하도록 했다.

4.3 환경 분류 — 실내/실외 임계 분리

실외는 EMPTY/PARKED의 NCC 분리가 명확하지만 실내는 좁다. 하나의 임계값으로 모두 커버할 수 없다고 판단해, **학습 시 환경(실내/실외)을 선택**하고 그에 맞는 임계값을 자동 적용하도록 했다.

4.4 학습 자기 일관성 검사

라이브 학습 도중 차·사람이 지나가면 reference가 오염된다. 학습 후 수집 프레임들의 상호 NCC 평균/최솟값을 계산해 임계 이하이면 **경고 후 재학습을 유도**, 오염된 모델이 저장되는 것을 방지했다.

5. 시스템 구성

실시간 감지 GUI — WebSocket으로 다중 카메라 프레임을 받아, 첫 프레임에서 학습된 reference(**npz**)를 자동 로드하고 환경별 임계값을 적용. 매 프레임 NCC·히스테리시스·안정성 검증을 거쳐 입출차 이벤트를 CSV로 로깅하고 카메라별 카드 UI에 표시. 라이브 학습 (환경 선택 + 자기 일관성 검사) 기능 포함.

오프라인 분석 GUI — tmap 파일을 불러와 캔버스에서 ROI를 직접 그리고, 빈 구간을 지정해 baseline을 학습. σ 맵 기반 **$occ = 1 - NCC/B$** 판정을 3단 그래프와 이벤트 리스트로 시각화. ROI 불변 판정을 실험·검증하는 도구.

학습 모델 포맷(npz**)** — 정규화 reference, σ 맵, 환경 정보, 메타데이터를 함께 저장해 워밍업 없이 즉시 로드.